

Assignment 3 : Trajectory Optimization in Joint Space

1. Problem : 2-link planar arm

State variables:

$$q = [q_1(t_0), q_2(t_0), q_1(t_1), q_2(t_1), \dots, q_1(t_N), q_2(t_N)]$$

- Time is discretized into N steps from 0→T
- Decision variables are the joint angles at each time step

2. Cost function :

Minimize the sum of squared joint accelerations :

$$J = \sum_{k=1}^{N-1} \left(\ddot{q}_1(k)^2 + \ddot{q}_2(k)^2 \right)$$

3. Constraints :

$$q_1(0) = q_{1,\text{start}}, \quad q_2(0) = q_{2,\text{start}}$$

$$q_1(T) = q_{1,\text{end}}, \quad q_2(T) = q_{2,\text{end}}$$

optimal joint limits

$$q_{i,\min} \leq q_i(t_k) \leq q_{i,\max}, \quad \forall k=0,1,\dots,N, \quad i=1,2$$

4. Python Code :

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.optimize import minimize

# Time parameters

T = 5.0

N = 50

dt = T / (N - 1)

# Start and end joint angles

q_start = np.array([0.0, 0.0])

q_end = np.array([np.pi/2, np.pi/3])

# Cost function

def cost_function(q):

    q = q.reshape(N, 2)

    q_ddot = np.diff(q, n=2, axis=0) / dt**2

    return np.sum(q_ddot[:,0]**2 + q_ddot[:,1]**2)

# Constraints

constraints = [

    {'type': 'eq', 'fun': lambda q: q[0] - q_start[0]},

    {'type': 'eq', 'fun': lambda q: q[1] - q_start[1]},
```

```

{'type': 'eq', 'fun': lambda q: q[-2] - q_end[0]},

{'type': 'eq', 'fun': lambda q: q[-1] - q_end[1]},

]

# Initial guess

q1_guess = np.linspace(q_start[0], q_end[0], N)

q2_guess = np.linspace(q_start[1], q_end[1], N)

q_init = np.vstack((q1_guess, q2_guess)).T.flatten()

# Optimization

result = minimize(cost_function, q_init, constraints=constraints,

                  method='SLSQP', options={'maxiter': 1000})

q_opt = result.x.reshape(N, 2)

# Time vector

time = np.linspace(0, T, N)

# Polynomial trajectory

q1_poly = q_start[0] + (q_end[0] - q_start[0]) * (3*(time/T)**2 - 2*(time/T)**3)

q2_poly = q_start[1] + (q_end[1] - q_start[1]) * (3*(time/T)**2 - 2*(time/T)**3)

# Plot

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)

```

```
plt.plot(time, q_opt[:,0], label='Optimized')

plt.plot(time, q1_poly, '--', label='Polynomial')

plt.xlabel('Time (s)')

plt.ylabel('q1 (rad)')

plt.legend()

plt.title('Joint 1')
```

```
plt.subplot(1,2,2)

plt.plot(time, q_opt[:,1], label='Optimized')

plt.plot(time, q2_poly, '--', label='Polynomial')

plt.xlabel('Time (s)')

plt.ylabel('q2 (rad)')

plt.legend()

plt.title('Joint 2')
```

```
plt.tight_layout()

plt.show()
```

```
# Cost comparison
```

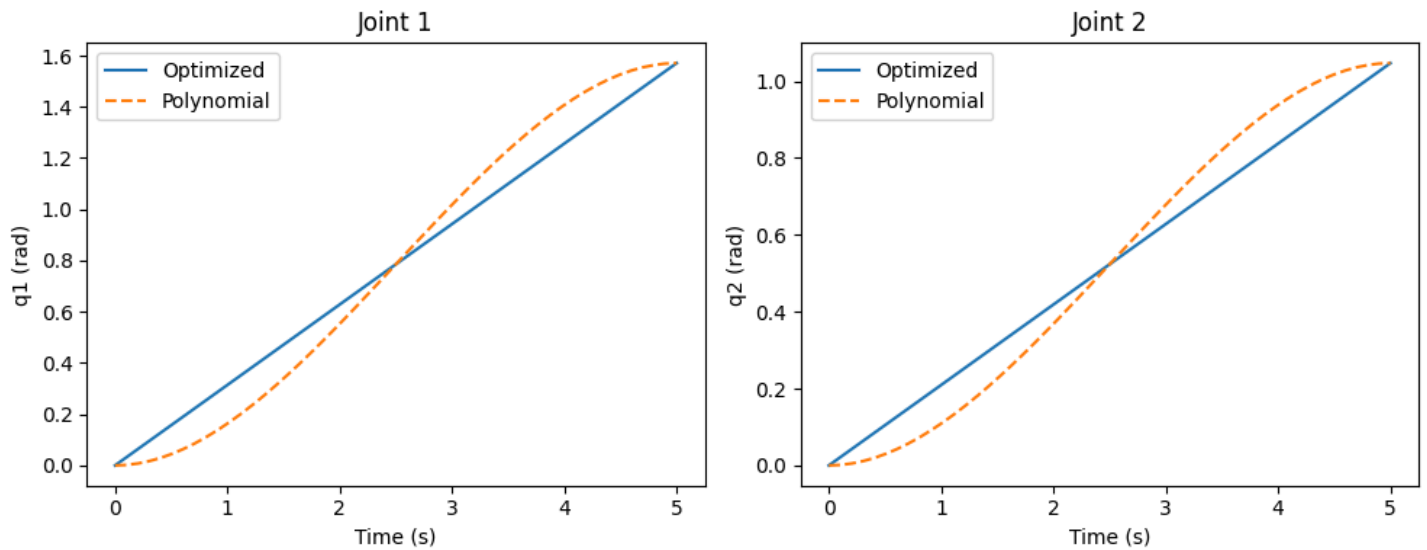
```
cost_poly = cost_function(np.vstack((q1_poly, q2_poly)).T.flatten())
```

```
cost_opt = cost_function(q_opt.flatten())
```

```
print("Polynomial trajectory cost:", cost_poly)
```

```
print("Optimized trajectory cost:", cost_opt)
```

5. Images :



Polynomial trajectory cost: 3.150538861226121

Optimized trajectory cost: 8.317874051173495e-13

6. Discussions :

The joint space trajectories are produced using numerical optimization rather than polynomials. This optimal trajectory was produced based on minimizing the sum of squared values of joint accelerations. Based on this method, the optimal trajectory is smooth compared to the other trajectory produced by polynomials. It is evident from the joint angle plots that the optimal trajectory has a lower curve and fewer abrupt changes. The cost function for the optimal trajectory is lower compared to the cost function of the polynomial trajectory. The smooth trajectory helps in reducing stresses on the actuator and mechanical components in a robot. It is easy to add another constraint to the trajectory optimization problem.